Data Mining
Classification: Alternative Techniques

Lecture Notes for Chapter 4

Rule-Based

Introduction to Data Mining , 2nd Edition

by

Tan, Steinbach, Karpatne, Kumar

Rule-Based Classifier

- Classify records by using a collection of "if…then…" rules
- Rule: (*Condition*) -> *y*
  - where
    - *Condition* is a conjunction of tests on attributes
    - *y* is the class label
  - Examples of classification rules:
    - (Blood Type=Warm) and (Lay Eggs=Yes) -> Birds
    - (Taxable Income < 50K) and (Refund=Yes) -> Evade=No

Rule-based Classifier (Example)

Name	Blood Type	Give Birth	Can Fly	Live in Water	Class
human	warm	yes	no	no	mammals
python	cold	no	no	no	reptiles
salmon	cold	no	no	yes	fishes
whale	warm	yes	no	yes	mammals
frog	cold	no	no	sometimes	amphibians
komodo	cold	no	no	no	reptiles
bat	warm	yes	yes	no	mammals
pigeon	warm	no	yes	no	birds
cat	warm	yes	no	no	mammals
leopard shark	cold	yes	no	yes	fishes
turtle	cold	no	no	sometimes	reptiles
penguin	warm	no	no	sometimes	birds
porcupine	warm	yes	no	no	mammals
eel	cold	no	no	yes	fishes
salamander	cold	no	no	sometimes	amphibians
gila monster	cold	no	no	no	reptiles
platypus	warm	no	no	no	mammals
owl	warm	no	yes	no	birds
dolphin	warm	yes	no	yes	mammals
eagle	warm	no	yes	no	birds

R1: (Give Birth = no) and (Can Fly = yes) => Birds

R2: (Give Birth = no) and (Live in Water = yes) => Fishes

R3: (Give Birth = yes) and (Blood Type = warm) => Mammals

R4: (Give Birth = no) and (Can Fly = no) => Reptiles

R5: (Live in Water = sometimes) => Amphibians

Application of Rule-Based Classifier

- A rule *r* covers an instance **x** if the attributes of the instance satisfy the condition of the rule

R1: (Give Birth = no) and (Can Fly = yes) => Birds

R2: (Give Birth = no) and (Live in Water = yes) => Fishes

R3: (Give Birth = yes) and (Blood Type = warm) => Mammals

R4: (Give Birth = no) and (Can Fly = no) => Reptiles

R5: (Live in Water = sometimes) => Amphibians

Name	Blood Type	Give Birth	Can Fly	Live in Water	Class
hawk	warm	no	yes	no	?
grizzly bear	warm	yes	no	no	?

The rule R1 covers a hawk => Bird

The rule R3 covers the grizzly bear => Mammal

Rule Coverage and Accuracy

- Coverage of a rule:
  - Fraction of records that satisfy the antecedent of a rule
- Accuracy of a rule:
  - Fraction of records that satisfy the antecedent that also satisfy the consequent of a rule

Tid	Refund	Marital Status	Taxable Income	Class
1	Yes	**Single**	125K	**No**
2	No	Married	100K	**No**
3	No	**Single**	70K	**No**
4	Yes	Married	120K	**No**
5	No	Divorced	95K	**Yes**
6	No	Married	60K	**No**
7	Yes	Divorced	220K	**No**
8	No	**Single**	85K	**Yes**
9	No	Married	75K	**No**
10	No	**Single**	90K	**Yes**

(Status=Single) => No

Coverage = 40%, Accuracy = 50%

Terminology

- If a rule is satisfied by X, the rule is said to be **triggered**.
- If R is the only rule satisfied, then the rule **fires** by returning the class prediction for X.

Note that triggering does not always mean firing because there may be more than one rule that is satisfied! If more than one rule is triggered, we have a potential problem. What if they each specify a different class? Or what if no rule is satisfied by X?

How does Rule-based Classifier Work?

R1: (Give Birth = no) and (Can Fly = yes) => Birds

R2: (Give Birth = no) and (Live in Water = yes) => Fishes

R3: (Give Birth = yes) and (Blood Type = warm) => Mammals

R4: (Give Birth = no) and (Can Fly = no) => Reptiles

R5: (Live in Water = sometimes) => Amphibians

Name	Blood Type	Give Birth	Can Fly	Live in Water	Class
lemur	warm	yes	no	no	?
turtle	cold	no	no	sometimes	?
dogfish shark	cold	yes	no	yes	?

A lemur triggers rule R3, so it is classified as a mammal

A turtle triggers both R4 and R5

A dogfish shark triggers none of the rules

Characteristics of Rule Sets: Strategy 1

- Mutually exclusive rules
  - Classifier contains mutually exclusive rules if the rules are independent of each other
  - Every record is covered by at most one rule

- Exhaustive rules
  - Classifier has exhaustive coverage if it accounts for every possible combination of attribute values
  - Each record is covered by at least one rule

Characteristics of Rule Sets: Strategy 2

- Rules are not mutually exclusive
  - A record may trigger more than one rule
  - Solution?

    - Unordered rule set – a test record may trigger multiple classification rules. Use voting schemes to determine the class label of the record.

    - Ordered rule set
- Rules are not exhaustive
  - A record may not trigger any rules
  - Solution?

    -Use a default class

Ordered Rule Set

- Rules are rank ordered according to their priority. Priority can be defined in different ways (based on accuracy, coverage, total description length, order of rule generation, give attributes weight by importance etc)
  - An ordered rule set is known as a decision list
- When a test record is presented to the classifier
  - It is assigned to the class label of the highest ranked rule it has triggered
  - If none of the rules fired, it is assigned to the default class

R1: (Give Birth = no) and (Can Fly = yes) => Birds

R2: (Give Birth = no) and (Live in Water = yes) => Fishes

R3: (Give Birth = yes) and (Blood Type = warm) => Mammals

R4: (Give Birth = no) and (Can Fly = no) => Reptiles

R5: (Live in Water = sometimes) => Amphibians

Name	Blood Type	Give Birth	Can Fly	Live in Water	Class
turtle	cold	no	no	sometimes	?

Rule Ordering Schemes

- Rule-based ordering
  - Individual rules are ranked based on their quality
- Class-based ordering
  - Rules that belong to the same class appear together, nay be sorted based on the class information, but in class ordering is not important. We will focus on this type of ordering scheme

Rule-based Ordering

(Refund=Yes) ==> No

(Refund=No, Marital Status={Single,Divorced}, Taxable Income<80K) ==> No

(Refund=No, Marital Status={Single,Divorced}, Taxable Income>80K) ==> Yes

(Refund=No, Marital Status={Married}) ==> No

Class-based Ordering

(Refund=Yes) ==> No

(Refund=No, Marital Status={Single,Divorced}, Taxable Income<80K) ==> No

(Refund=No, Marital Status={Married}) ==> No

(Refund=No, Marital Status={Single,Divorced}, Taxable Income>80K) ==> Yes

Building Classification Rules

- Direct Method:

  - Extract rules directly from data, which identify key relationships between the attributes of a data set and a class label.

  - Examples: RIPPER, CN2, Holte's 1R

- Indirect Method:

  - Extract rules from other classification models (e.g. decision trees, neural networks, etc).

  - Examples: C4.5rules

Direct Method: Sequential Covering

1. Start from an empty rule
2. Grow a rule using the Learn-One-Rule function (should extract the best rule for a class y that covers the set of training records)
3. Remove training records covered by the rule
4. Repeat Step (2) and (3) until stopping criterion is met

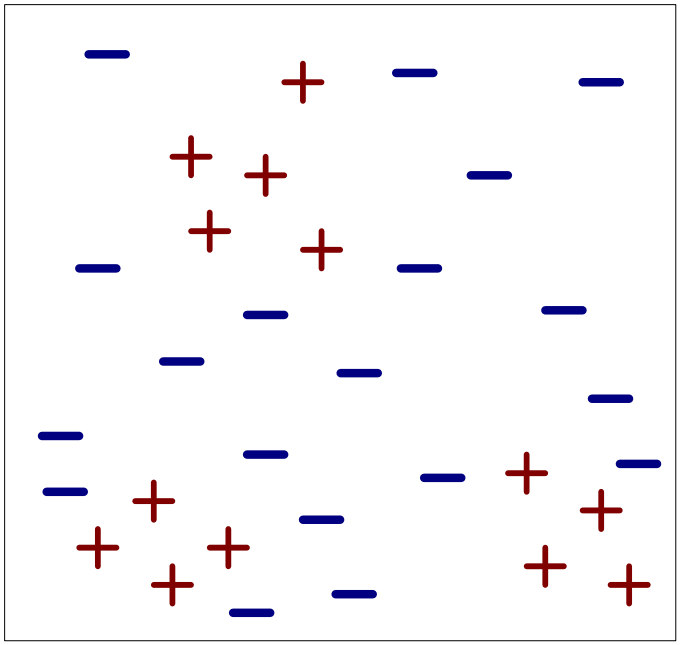
Rules are defined based on a certain evaluation measure. The algorithm extracts the rules one class at a time. The criterion for deciding which class should be generated first depends on number of factors (class prevalence or the cost of misclassifying records from a class)

Example of Sequential Covering

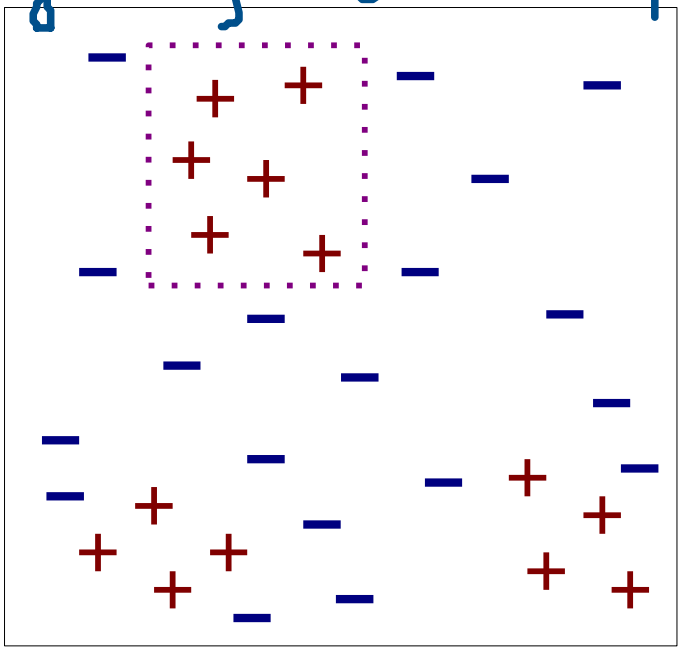
During rule extraction, all training records from class y are considered to be positive examples, while everything else is considered to be negative (for all the rest of the classes)

- desirable rule – covers most of the positive examples and none (or few of the negative once)

largest fraction of +



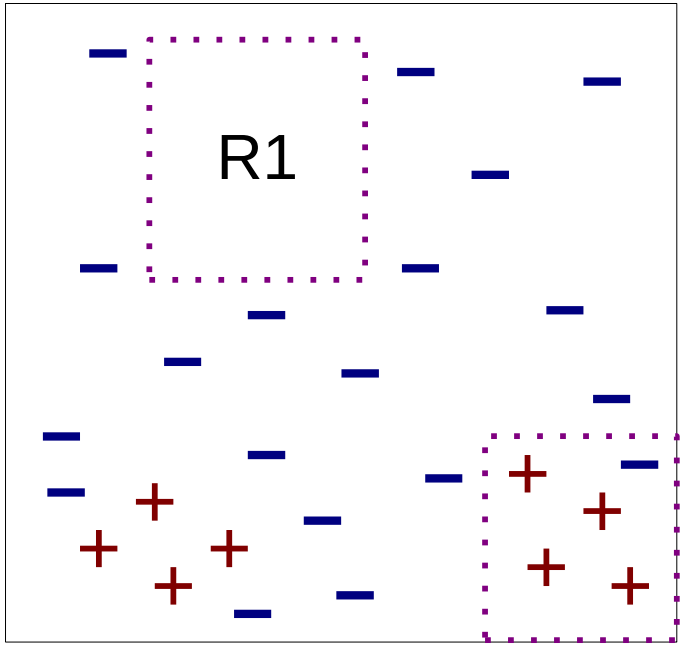
(i) Original Data



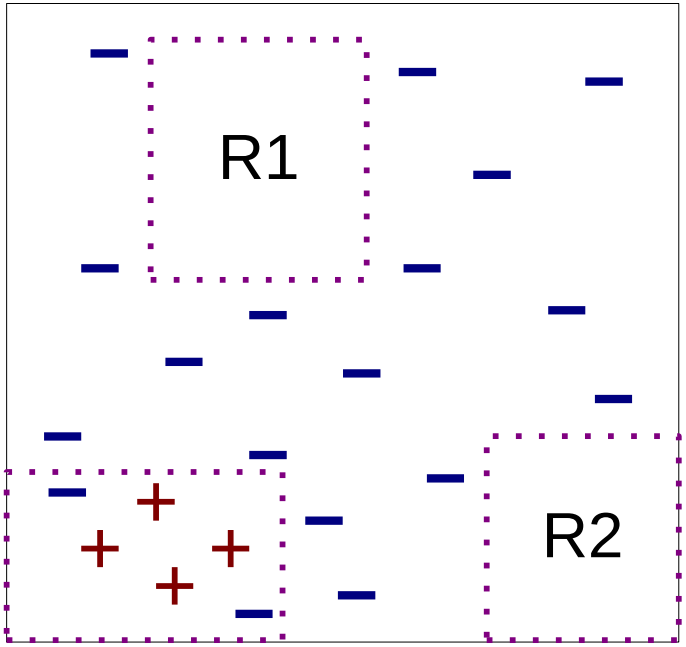
(ii) Step 1

Example of Sequential Covering…

Training data cover by R1 are removed, R1 is added to the bottom of the decision list R. Then we look for the second best rule for the class etc.



(iii) Step 2

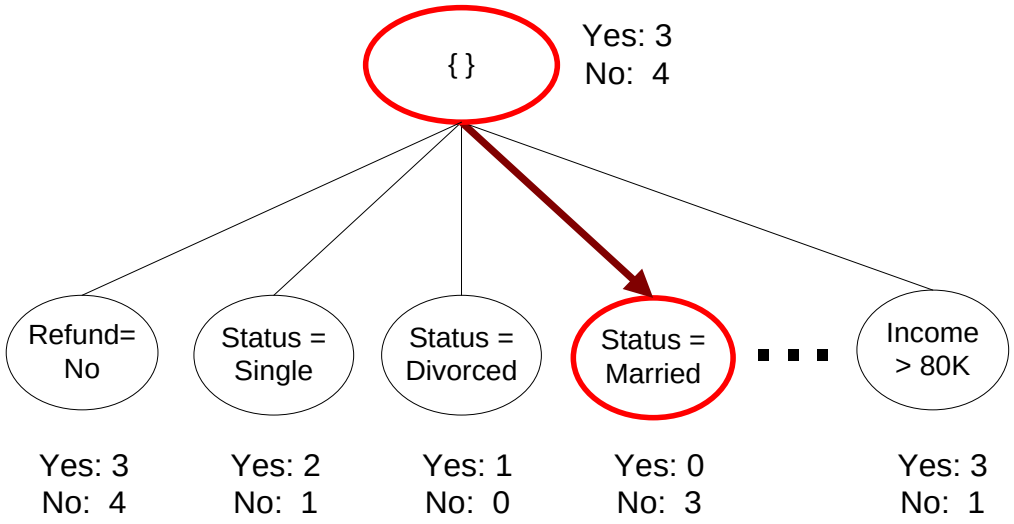


(iv) Step 3

Rule Growing

Two common strategies

- **General to specific** - start from initial {}=>y (empty set=>target class) than adds conjuncts which improve the rule's quality, until some stopping criterion is met (when the added conjunct does not improve the quality, max length of the conjuncts set etc

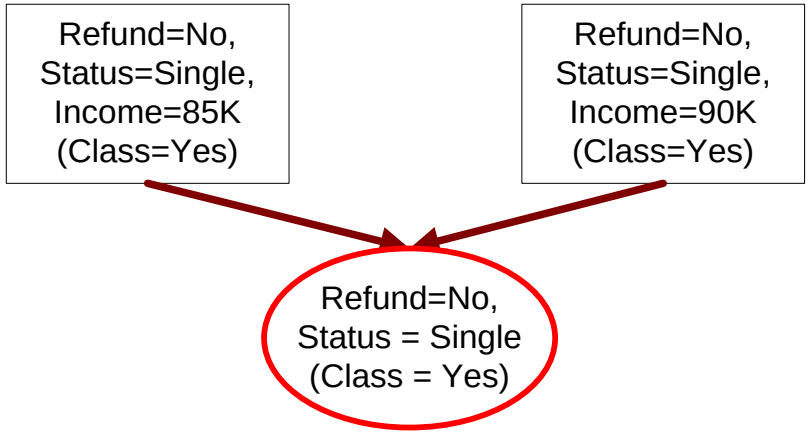


(a) General-to-specific

Rule Growing

Two common strategies

Specific to general - one positive example is chosen at random and its attribute values are set to be the initial rule (this is a specific example), which is then generalized by removing one of its conjuncts, so that it can cover more positive examples. Refinement step is repeated until stopping criterion is met (ex. The rule starts to cover negative examples)



(b) Specific-to-general

Rule Evaluation

How to determine which conjunct should be added or removed – we need an evaluation metric

- accuracy measure – does not take into account the rule's coverage.

Possible approaches

- statistical test to prune (discard) rules with poor coverage (likelihood ratio statistics)
- evaluation metric that takes into account the rule coverage (Laplace, m-estimate)
- evaluation metric that takes into account the support count of the rule. Ex. FOIL's information gain

- Foil's Information Gain

FOIL: First Order Inductive Learner – an early rule-based learning algorithm

  - R0: {} => class (initial rule)
  - R1: {A} => class (rule after adding conjunct)
  - $Gain(R_0, R_1) = p_1 \times [\, log_2\left(\frac{p_1}{p_1 + n_1}\right) - log_2\left(\frac{p_0}{p_0 + n_0}\right)]$
  - p_0: number of positive instances covered by R0
    n_0: number of negative instances covered by R0
    p_1: number of positive instances covered by R1
    n_1: number of negative instances covered by R1

Support count corresponds to the number of positive examples covered by the rule. The measure is proportional to p1 and p1/(p1+n1. Thus the measure prefers rules that have high support and accuracy)

Direct Method: RIPPER

- For 2-class problem, choose one of the classes as positive class, and the other as negative class
  - Learn rules for positive class
  - Negative class will be default class
- For multi-class problem
  - Order the classes according to increasing class prevalence (fraction of instances that belong to a particular class)
  - Learn the rule set for smallest class first, treat the rest as negative class
  - Repeat with next smallest class as positive class

Direct Method: RIPPER

Growing a rule (uses general to specific strategy):

- Start from empty rule
- Add conjuncts as long as they improve FOIL's information gain
- Stop adding conjuncts when the rule starts covering negative examples
- Prune the rule immediately using incremental reduced error pruning
- Measure for pruning: v = (p-n)/(p+n)

  p (respectively n) is the number of positive (respectively negative) examples covered by the rule in the validation set

If the metric improves after pruning, the conjunct is removed. This is done the following way - The pruning starts on the last conjunct ex. ABCD->y, RIPPER checks whether D should be pruned first, then CD, BCD etc. Then delete any final sequence of conditions that maximizes v.

Direct Method: RIPPER

Building a Rule Set:

- Use sequential covering algorithm

- Finds the best rule that covers the current set of positive examples

- Eliminate both positive and negative examples covered by the rule

- Each time a rule is added to the rule set, compute the new description length. Stop adding new rules when the new description length is d bits longer than the smallest description length obtained so far (minimum description length principle MDL). Another possible stopping condition is that the error rate of the rule on the validation set must not exceed 50%.

Direct Method: RIPPER

RIPPER also performs optimization method to see if there are better alternative rules.

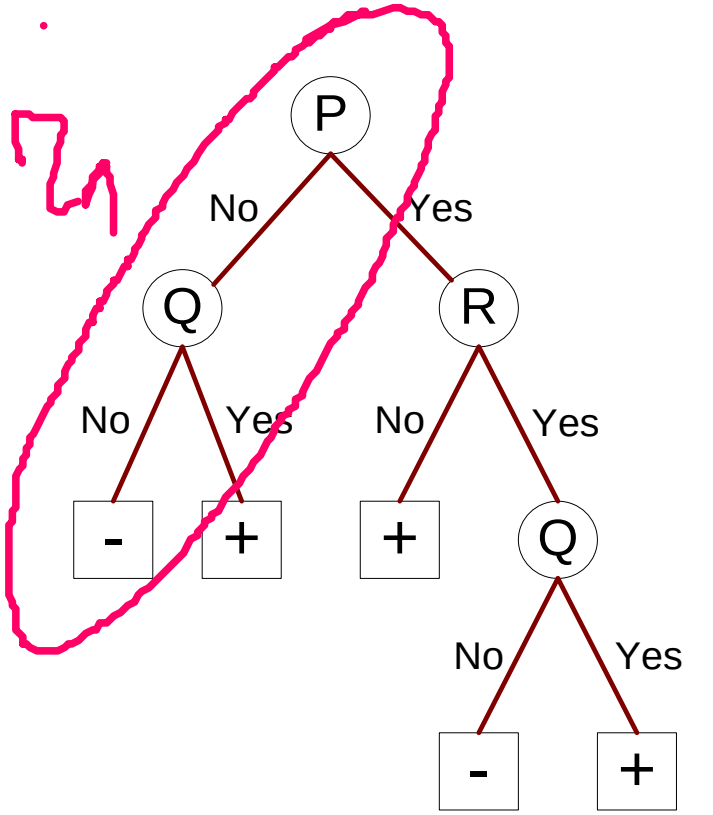
Optimize the rule set: For each rule *r* in the rule set ***R***

- Consider 2 alternative rules:
  - Replacement rule (r*): grow new rule from scratch
  - Revised rule (r′): add conjuncts to extend the rule *r*
- Compare the rule set for *r* against the rule set for r* and r′
- Choose rule set that minimizes MDL principle

- Repeat rule generation and rule optimization for the remaining positive examples

Indirect Methods

Method for generating a rule set from a decision tree. Every path from the root to a leaf in a decision tree can be expresses as a classification rule.



Rule Set

r1: (P=No,Q=No) ==> -
r2: (P=No,Q=Yes) ==> +
r3: (P=Yes,R=No) ==> +
r4: (P=Yes,R=Yes,Q=No) ==> -
r5: (P=Yes,R=Yes,Q=Yes) ==> +

Indirect Method: C4.5rules

- Extract rules from an unpruned decision tree
- For each rule, r: A => y,
  - consider an alternative rule r′: A′ => y where A′ is obtained by removing one of the conjuncts in A
  - Compare the pessimistic error rate for r against all r's
  - Prune if one of the alternative rules has lower pessimistic error rate
  - Repeat until we can no longer improve generalization error
  - Discard any duplicate rule, if there is any

Indirect Method: C4.5rules

Instead of ordering the rules, order subsets of rules (class ordering)

- Each subset is a collection of rules with the same rule consequent (class)

- Compute description length of each subset

  - Description length = L(error) + g L(model)

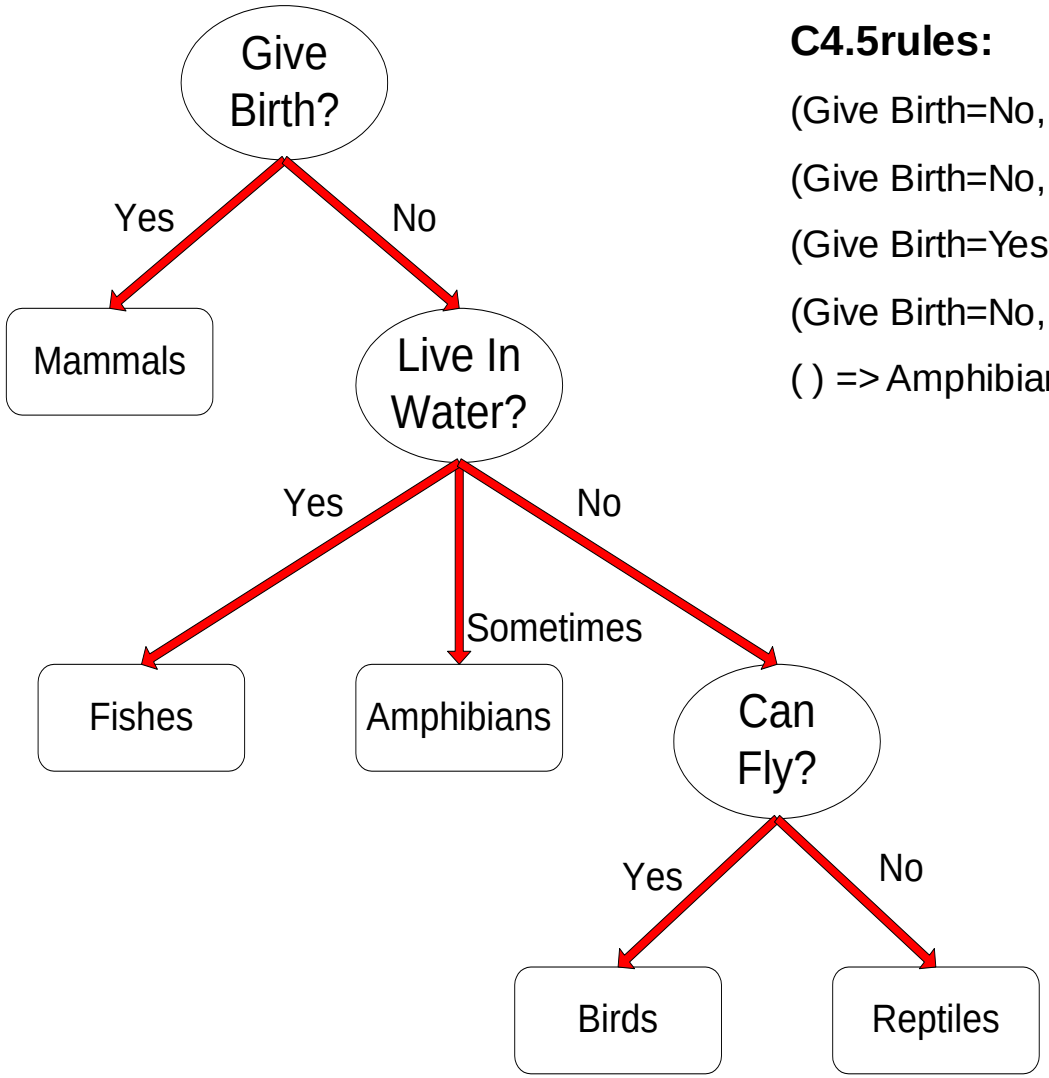
  - g is a parameter that takes into account the presence of redundant attributes in a rule set (default value = 0.5)

Order classes in increasing order of their total description length. The first one is given the highest priority.

Example

Name	Give Birth	Lay Eggs	Can Fly	Live in Water	Have Legs	Class
human	yes	no	no	no	yes	mammals
python	no	yes	no	no	no	reptiles
salmon	no	yes	no	yes	no	fishes
whale	yes	no	no	yes	no	mammals
frog	no	yes	no	sometimes	yes	amphibians
komodo	no	yes	no	no	yes	reptiles
bat	yes	no	yes	no	yes	mammals
pigeon	no	yes	yes	no	yes	birds
cat	yes	no	no	no	yes	mammals
leopard shark	yes	no	no	yes	no	fishes
turtle	no	yes	no	sometimes	yes	reptiles
penguin	no	yes	no	sometimes	yes	birds
porcupine	yes	no	no	no	yes	mammals
eel	no	yes	no	yes	no	fishes
salamander	no	yes	no	sometimes	yes	amphibians
gila monster	no	yes	no	no	yes	reptiles
platypus	no	yes	no	no	yes	mammals
owl	no	yes	yes	no	yes	birds
dolphin	yes	no	no	yes	no	mammals
eagle	no	yes	yes	no	yes	birds

C4.5 versus C4.5rules versus RIPPER



C4.5rules:

(Give Birth=No, Can Fly=Yes) => Birds

(Give Birth=No, Live in Water=Yes) => Fishes

(Give Birth=Yes) => Mammals

(Give Birth=No, Can Fly=No, Live in Water=No) =>Reptiles

( ) => Amphibians

RIPPER:

(Live in Water=Yes) → Fishes

(Have Legs=No) → Reptiles

(Give Birth=No, Can Fly=No, Live In Water=No) → Reptiles

(Can Fly=Yes,Give Birth=No) → Birds

() → Mammals

C4.5 versus C4.5rules versus RIPPER

C4.5 and C4.5rules:

			PREDICTED CLASS			
		Amphibians	Fishes	Reptiles	Birds	Mammals
ACTUAL CLASS	Amphibians	2	0	0	0	0
	Fishes	0	2	0	0	1
	Reptiles	1	0	3	0	0
	Birds	1	0	0	3	0
	Mammals	0	0	1	0	6

RIPPER:

			PREDICTED CLASS			
		Amphibians	Fishes	Reptiles	Birds	Mammals
ACTUAL CLASS	Amphibians	0	0	0	0	2
	Fishes	0	3	0	0	0
	Reptiles	0	0	3	0	1
	Birds	0	0	1	2	1
	Mammals	0	2	1	0	4

Advantages of Rule-Based Classifiers

- Has characteristics quite similar to decision trees
  - As highly expressive as decision trees
  - Easy to interpret
  - Performance comparable to decision trees
  - Can handle redundant attributes
- Better suited for handling imbalanced classes
- Harder to handle missing values in the test set